

Operating Systems Study Guide

PROCESS SYNCHRONIZATION & DEADLOCKS Q&A

1. What is Critical Section? What are the requirements for a solution to the Critical Section Problem?

A critical section is the part of a process where shared data or resources are accessed. Since multiple processes may access the same resource simultaneously, synchronization is required to avoid race conditions and maintain data consistency.

Critical Section

When a process enters its critical section, it performs operations on shared resources such as memory, files, variables, or devices. If two or more processes execute their critical sections at the same time, incorrect results may occur due to concurrent access.

Requirements for a Solution to the Critical Section Problem

Mutual Exclusion

Mutual exclusion ensures that only one process can execute in its critical section at any given time. If one process is accessing a shared resource, all other processes must wait until the resource becomes available.

Progress

Progress guarantees that if no process is in its critical section and some processes wish to enter, the choice of the next process should not be delayed indefinitely. Only processes that want to enter the critical section should participate in the decision.

Bounded Waiting

Bounded waiting ensures that after a process requests entry into its critical section, there is a limit on the number of times other processes can enter before its request is granted. This prevents starvation and ensures fairness among processes.

2. Explain the Concept of Semaphores in Process Synchronization.

A semaphore is a synchronization mechanism used to control access to shared resources by multiple processes. It helps coordinate process execution and prevents conflicts when resources are shared.

Semaphore Operations

wait() Operation

The wait() operation decreases the semaphore value by one. If the resulting value is negative, the process is blocked and placed in a waiting queue until the required resource becomes available.

signal() Operation

The signal() operation increases the semaphore value by one. If one or more processes are waiting, one of them is awakened and allowed to continue execution.

Types of Semaphores

Binary Semaphore

A binary semaphore can have only two values, 0 and 1. It is used to provide mutual exclusion when only one process should access a resource at a time.

Counting Semaphore

A counting semaphore can take any non-negative integer value. It is useful when multiple instances of a resource are available and need to be managed.

Applications

Semaphores are widely used in synchronization problems such as Producer-Consumer, Readers-Writers, and Dining Philosophers. They help ensure safe and efficient sharing of resources.

3. Explain Bounded-Buffer Problem.

The Bounded-Buffer Problem, also known as the Producer-Consumer Problem, is a synchronization problem involving a producer, a consumer, and a shared buffer of limited size.

Producer

The producer process generates data items and places them into the shared buffer. It can continue producing data only if free space is available in the buffer.

Consumer

The consumer process removes data items from the buffer and uses them. It can consume data only when the buffer contains one or more items.

Problems in the System

Buffer Full

If the buffer becomes full, the producer cannot insert additional items. It must wait until the consumer removes some items and creates free space.

Buffer Empty

If the buffer becomes empty, the consumer cannot remove any item. It must wait until the producer places new items into the buffer.

Simultaneous Access

If both producer and consumer access the buffer simultaneously, data inconsistency may occur. Therefore, proper synchronization is necessary.

Solution

The problem is solved using semaphores such as mutex, empty, and full. These semaphores ensure mutual exclusion and proper coordination between producer and consumer processes.

4. What is Readers-Writers Problem? Explain.

The Readers-Writers Problem is a synchronization problem in which several processes share a common data resource. Some processes read the data, while others modify it.

Readers

Readers access the shared data only for reading purposes. Since they do not modify the data, multiple readers can read the resource simultaneously.

Writers

Writers modify the shared data. To maintain data consistency, only one writer is allowed to access the resource at a time.

Conditions of the Problem

Multiple Readers Allowed

Several readers can access the data concurrently without affecting one another. This improves system performance and resource utilization.

Writer Exclusiveness

When a writer is writing to the shared resource, no other writer or reader is allowed to access it. This prevents inconsistent or incomplete data.

Issues

Writer Starvation

If readers continuously access the resource, a writer may be forced to wait indefinitely for access.

Reader Starvation

In some solutions where writers are given higher priority, readers may also experience long waiting times.

5. Explain Dining Philosophers Problem.

The Dining Philosophers Problem is a classical synchronization problem that demonstrates deadlock and resource allocation issues in operating systems.

Problem Description

Five philosophers sit around a circular table with one chopstick placed between every pair of philosophers. Each philosopher alternates between thinking and eating.

To eat, a philosopher must acquire both the left and right chopsticks. After eating, the philosopher releases the chopsticks and returns to thinking.

Deadlock

If all philosophers pick up their left chopstick simultaneously, each philosopher waits for the right chopstick. Since every philosopher is waiting for a resource held by another philosopher, none can proceed.

This results in a deadlock where all philosophers remain blocked indefinitely.

Starvation

A philosopher may repeatedly fail to obtain both chopsticks while neighboring philosophers continue eating. As a result, the philosopher may never get an opportunity to eat.

Solutions

Limiting the Number of Philosophers

Only four philosophers are allowed to attempt eating simultaneously. This reduces the possibility of circular waiting and deadlock.

Resource Ordering

Philosophers are required to pick up chopsticks in a specific order. This breaks the circular wait condition and prevents deadlock.

Semaphore Solution

Semaphores can be used to control access to chopsticks and ensure proper synchronization among philosophers.

6. Explain Any One Classic Problem of Synchronization.

Producer-Consumer Problem

The Producer-Consumer Problem is a classical synchronization problem involving two processes that share a common buffer. It demonstrates the need for coordination when multiple processes access shared resources.

Producer Process

The producer generates data items and inserts them into the shared buffer. It must ensure that sufficient space is available before producing new items.

Consumer Process

The consumer removes data items from the buffer and processes them. It can consume data only when the buffer contains valid items.

Problems

Buffer Overflow

If the producer continues inserting data when the buffer is already full, overflow occurs and data may be lost.

Buffer Underflow

If the consumer attempts to remove data from an empty buffer, underflow occurs because no data is available for processing.

Race Condition

If both producer and consumer access the buffer simultaneously without synchronization, inconsistent results may occur.

Solution

Semaphores such as mutex, empty, and full are used to synchronize producer and consumer activities. These semaphores ensure mutual exclusion and proper coordination, allowing the buffer to be used safely and efficiently.

7. What are the Necessary Conditions for Deadlock Situation to Occur?

A deadlock is a situation in which two or more processes are unable to proceed because each process is waiting for a resource held by another process. For a deadlock to occur, four necessary conditions must hold simultaneously.

Mutual Exclusion

At least one resource must be non-shareable. Only one process can use the resource at a time, and other processes requesting it must wait.

Hold and Wait

A process holding one or more resources is allowed to request additional resources. While waiting for new resources, it continues to hold the resources already allocated to it.

No Preemption

Resources cannot be forcibly taken away from a process. A resource can be released only voluntarily by the process holding it after completing its task.

Circular Wait

A set of processes exists such that each process is waiting for a resource held by the next process in the cycle. This circular chain of waiting leads to deadlock.

8. Explain Deadlock Prevention.

Deadlock prevention is a technique used to ensure that at least one of the necessary conditions for deadlock never occurs. By eliminating one or more conditions, deadlocks can be prevented.

Eliminating Mutual Exclusion

Deadlock can be prevented if resources are made shareable whenever possible. However, some resources such as printers cannot be shared and require mutual exclusion.

Eliminating Hold and Wait

A process is required to request all necessary resources before execution begins. If all resources are not available, the process waits without holding any resources.

Eliminating No Preemption

If a process requests a resource that is unavailable, the resources currently held by the process can be released and allocated to other processes. The process can request them again later.

Eliminating Circular Wait

A fixed ordering is assigned to all resource types. Processes must request resources only in increasing order of numbering, preventing circular waiting.

9. Explain Resource Allocation Graph with Example.

A Resource Allocation Graph (RAG) is a graphical method used to represent the allocation of resources to processes and resource requests in a system. It helps in analyzing deadlocks.

Components of a Resource Allocation Graph

Process Node

Processes are represented by circles. Each circle corresponds to a process in the system.

Resource Node

Resources are represented by rectangles. Each rectangle corresponds to a resource type.

Request Edge

A directed edge from a process to a resource indicates that the process is requesting that resource. Example: $P1 \rightarrow R1$

Assignment Edge

A directed edge from a resource to a process indicates that the resource has been allocated to that process. Example: $R1 \rightarrow P1$

Deadlock Detection Using RAG

If the graph contains no cycle, deadlock does not exist.

If the graph contains a cycle and each resource has only one instance, deadlock exists.

Example

Suppose: Process P1 holds resource R1 and requests R2. Process P2 holds resource R2 and requests R1.

The graph contains the cycle: $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$. Since a circular wait exists, the system is in a deadlock state.

10. Explain Deadlock Avoidance using Safe State Algorithm.

Deadlock avoidance ensures that the system never enters a deadlock state. It requires advance information about the maximum resource requirements of each process.

Safe State

A system is said to be in a safe state if there exists a sequence of process execution such that every process can obtain its required resources and complete successfully. This sequence is called a safe sequence.

Working of Safe State Algorithm

When a process requests resources, the operating system temporarily allocates them and checks whether the resulting state is safe.

Steps

1. Allocate the requested resources temporarily.
2. Determine the available resources.
3. Check whether all processes can complete in some order.
4. If a safe sequence exists, grant the request permanently.
5. If no safe sequence exists, deny the request and make the process wait.

Advantages

Prevents deadlocks before they occur. Ensures safe resource allocation. Maintains system stability.

11. Explain Deadlock Avoidance using Resource-Allocation-Graph Algorithm.

The Resource-Allocation-Graph Algorithm is a deadlock avoidance method used when each resource type has only one instance. It avoids deadlock by preventing the formation of cycles in the graph.

Claim Edge

A claim edge is represented by a dashed line from a process to a resource. It indicates that the process may request that resource in the future.

Resource Request

When a process requests a resource, the claim edge is converted into a request edge. Before granting the request, the system checks whether converting the edge would create a cycle.

Resource Allocation

If no cycle is formed, the resource is allocated and the request edge becomes an assignment edge. If a cycle would be created, the request is denied and the process must wait.

Working Principle

The algorithm continuously checks the graph and ensures that resource allocation never creates a cycle. Since cycles are avoided, deadlocks cannot occur.

12. Explain Banker's Algorithm.

Banker's Algorithm is a deadlock avoidance algorithm developed by Edsger W. Dijkstra. It determines whether granting a resource request will leave the system in a safe state.

Data Structures Used

Available: Represents the number of available instances of each resource type.

Max: Indicates the maximum number of resources each process may require.

Allocation: Shows the resources currently allocated to each process.

Need: Represents the remaining resource requirement of each process ($\text{Need} = \text{Max} - \text{Allocation}$).

Working of Banker's Algorithm

When a process requests resources, the operating system temporarily allocates them and checks whether the system remains in a safe state.

Steps

1. Verify that the request does not exceed the process's maximum claim.
2. Check whether the requested resources are available.
3. Temporarily allocate the resources.
4. Apply the safety algorithm.
5. If the system remains safe, grant the request permanently.
6. Otherwise, restore the previous state and make the process wait.

Advantages

Avoids deadlock effectively. Maintains system safety. Ensures efficient resource allocation.

13. Explain How to Recover from Deadlock.

Deadlock recovery is the process of removing a deadlock after it has been detected. The operating system must take corrective action to allow blocked processes to continue execution.

Process Termination

One method of recovery is to terminate processes involved in the deadlock.

Abort All Deadlocked Processes: All processes involved in the deadlock are terminated. This quickly removes the deadlock but may result in significant loss of work.

Abort One Process at a Time: Processes are terminated one by one until the deadlock cycle is eliminated. This approach reduces the amount of lost work.

Resource Preemption

Resources are taken away from selected processes and assigned to others until the deadlock is resolved.

Issues in Resource Preemption

Selecting a Victim: The operating system must decide which process should lose its resources based on priority, execution time, resources held, etc.

Rollback: After preemption, a process may be rolled back to a previous safe state and restarted later.

Starvation: The same process should not be repeatedly selected as a victim, otherwise it may never complete its execution.

14. Explain the Deadlock Detection Algorithm for Single Instance of Each Resource Type.

The Deadlock Detection Algorithm for Single Instance Resources uses a Resource Allocation Graph (RAG) to determine whether a deadlock exists in the system. Each resource type has only one instance.

Resource Allocation Graph consists of:

- Process nodes represented by circles.
- Resource nodes represented by rectangles.
- Request edges from a process to a resource.
- Assignment edges from a resource to a process.

Working of the Algorithm

The algorithm examines the Resource Allocation Graph and checks for the presence of cycles.

No Cycle in the Graph: If the graph contains no cycle, the system is not in a deadlock state. All processes can eventually obtain the resources they need and complete execution.

Cycle in the Graph: If the graph contains a cycle and each resource type has only one instance, then a deadlock exists.

Example: Consider two processes P1 and P2 and two resources R1 and R2. P1 holds R1 and requests R2. P2 holds R2 and requests R1. The graph contains the cycle: $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$. Since a cycle exists, the system is in a deadlock state.

15. Consider a System with 12 Magnetic Tape Drives and Three Processes P0, P1 and P2. Find a Safe Sequence.

Given

Process | Maximum Need | Allocation

P0 | 10 | 5

P1 | 4 | 2

P2 | 9 | 2

Total tape drives = 12

Step 1: Calculate Available Resources

Allocated Resources = $5 + 2 + 2 = 9$

Available Resources = Total Resources – Allocated Resources = $12 - 9 = 3$

Step 2: Calculate Need Matrix

Need = Max – Allocation

P0: $10 - 5 = 5$

P1: $4 - 2 = 2$

P2: $9 - 2 = 7$

Step 3: Find Safe Sequence

Available = 3

Execute P1: Need(P1) = $2 \leq 3$. P1 completes and releases 2 resources. New Available = $3 + 2 = 5$

Execute P0: Need(P0) = $5 \leq 5$. P0 completes and releases 5 resources. New Available = $5 + 5 = 10$

Execute P2: Need(P2) = $7 \leq 10$. P2 completes and releases 2 resources. New Available = $10 + 2 = 12$

Safe Sequence

P1 → P0 → P2. Therefore, the system is in a safe state and the safe sequence is: { P1, P0, P2 }

16. Explain the Resource Allocation Algorithm in Banker's Algorithm.

The Resource Allocation Algorithm determines whether a process's resource request can be granted immediately without causing the system to enter an unsafe state.

Step 1: Check Request $\leq$ Need

For process P_i , the requested resources must not exceed its remaining need. $Request_i \leq Need_i$. If this condition is false, the process has exceeded its maximum claim and an error is reported.

Step 2: Check Request $\leq$ Available

The requested resources must be available in the system. $Request_i \leq Available$. If sufficient resources are not available, the process must wait.

Step 3: Temporary Allocation

The system temporarily allocates the requested resources.

$Available = Available - Request_i$

$Allocation_i = Allocation_i + Request_i$

$Need_i = Need_i - Request_i$

Step 4: Apply Safety Algorithm

After temporary allocation, the safety algorithm is executed to determine whether the system remains in a safe state.

Step 5: Decision

If the system remains safe, the request is granted permanently. If the system becomes unsafe, the temporary allocation is cancelled and the process must wait.

17. Explain Banker's Safety Algorithm.

The Safety Algorithm is used in Banker's Algorithm to determine whether the system is in a safe state. It checks if there exists a sequence in which all processes can complete successfully.

Data Structures Used

Work: A temporary vector initialized with the Available resources.

Finish: A Boolean array initialized to False for all processes.

Step 1: Set $Work = Available$, $Finish[i] = False$ for all processes.

Step 2: Find a process P_i such that: $Finish[i] = False$ and $Need_i \leq Work$.

Step 3: If such a process exists, assume that it completes execution. Update: $Work = Work + Allocation_i$, $Finish[i] = True$.

Step 4: Repeat Step 2 until all processes are marked $Finish = True$ or no suitable process can be found.

Step 5: Decision

If all $Finish$ values become $True$, the system is in a safe state. Otherwise, the system is in an unsafe state.

Safe Sequence: The order in which processes are marked $Finish = True$ forms the safe sequence.

18. Find a Safe Sequence for the Given Allocation, Max, Need and Available Matrices.

Given Allocation Matrix:

P0 (0,1,0) | P1 (2,0,0) | P2 (3,0,2) | P3 (2,1,1) | P4 (0,0,2)

Given Need Matrix:

P0 (7,4,3) | P1 (1,2,2) | P2 (6,0,0) | P3 (0,1,1) | P4 (4,3,1)

Available = (3, 3, 2)

Step 1: Available = (3,3,2). Need(P1) = (1,2,2) $\leq$ (3,3,2). Execute P1. New Available = (3,3,2) + (2,0,0) = (5,3,2). Safe Sequence: P1

Step 2: Available = (5,3,2). Need(P3) = (0,1,1) $\leq$ (5,3,2). Execute P3. New Available = (5,3,2) + (2,1,1) = (7,4,3). Safe Sequence: P1 $\rightarrow$ P3

Step 3: Available = (7,4,3). Need(P4) = (4,3,1) $\leq$ (7,4,3). Execute P4. New Available = (7,4,3) + (0,0,2) = (7,4,5). Safe Sequence: P1 $\rightarrow$ P3 $\rightarrow$ P4

Step 4: Available = (7,4,5). Need(P0) = (7,4,3) $\leq$ (7,4,5). Execute P0. New Available = (7,4,5) + (0,1,0) = (7,5,5). Safe Sequence: P1 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P0

Step 5: Available = (7,5,5). Need(P2) = (6,0,0) $\leq$ (7,5,5). Execute P2. New Available = (7,5,5) + (3,0,2) = (10,5,7). Safe Sequence: P1 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P0 $\rightarrow$ P2

Safe Sequence: { P1, P3, P4, P0, P2 }. Hence, the system is in a safe state.